unfit to be solved with any greedy algorithm, because locally-optimal choices won't be possible anymore.

Negative-weight edges might seem counter-intuitive, but they are actually quite common: if we measure distances using the gas consumed to travel between two vertices, and the goal is "not ending up with an empty tank", then an edge corresponding to a road with a gas station could have a negative weight; likewise, if we associate a cost to the gas, then an edge that allows us to perform a second delivery, or a pick-up, could have negative cost because we could earn extra money if we travel it.

To cope with negative edges, we need to use *Bellman-Ford*'s algorithm, an ingenious algorithm that uses the dynamic programming technique to derive a solution that keeps into account negative-weight edges. Bellman-Ford's algorithm is more expensive to run than Dijkstra's: its running time is $O(|V|*|E|)$. Although it can be applied to a broader set of graphs, it too has some limitations: it can't work with graphs with negative-weight cycles[10] (at the same time, though, it can be used as a test to find such cycles).

14.5 Beyond Dijkstra: A*

As we have seen, BFS and Dijkstra's algorithms are very similar to each other; turns out, they both are a particular case of A* (pronounced *A-star*) algorithm.

This algorithm, shown in listing 14.13, is not just more generic: it improves the performance of Dijkstra in at least two different situations. Before delving into those scenarios,